

Tests avec Node.js

Tester une API Express de façon fiable et isolée

Module 3/7

Objectifs du module

Durée : 3h30

- Écrire des tests isolés et des tests asynchrones fiables
- Utiliser mocks et stubs pour découpler les dépendances
- Mettre en place des tests fonctionnels avec headless browsers
- Choisir entre Mocha, Jest et le runner natif `node:test`

Plan du module

1. Isolation des tests et tests asynchrones

- `async/await`, `done`, timeouts, hooks setup/teardown

2. Mocks et stubs

- sinon (spy/stub/mock), fakes de dépendances, injection, supertest

3. Tests fonctionnels avec headless browsers

- Playwright/Puppeteer, comparaison Mocha vs Jest, runner natif

1. Isolation des tests et tests asynchrones

Pourquoi isoler les tests

Concept clé : un test fiable est **déterministe**, **rapide** et **indépendant** des autres.

Un test isolé

- N'a pas d'effet de bord persistant
- Ne dépend pas de l'ordre d'exécution
- Recrée son état avant chaque exécution

Symptômes d'un test non isolé

- Passe seul, échoue en suite
- État partagé (variable globale, BDD)
- Faux positifs intermittents (flaky)

Tests asynchrones : async/await

La forme moderne et recommandée. Le runner attend la résolution de la promesse.

```
import { test } from "node:test";
import assert from "node:assert/strict";

test("récupère un utilisateur", async () => {
  const user = await getUser(1);
  assert.equal(user.id, 1);
});
```

Pas de `done`, pas de callback oublié : si la promesse rejette, le test échoue.

Tests asynchrones : le callback done

Hérité de Mocha. Utile pour des API à callbacks (events, streams) non promisifyées.

```
it("émet un événement ready", (done) => {  
  const server = createServer();  
  server.on("ready", () => {  
    assert.ok(server.listening);  
    done();           // signale la fin au runner  
  });  
  server.on("error", done); // propage l'erreur  
});
```

Piège : ne jamais mélanger `async` et `done` dans le même test.

Timeouts et tests lents

Concept clé : un test bloqué doit échouer vite, pas pendre la CI.

Mocha / Jest

```
it("appel réseau", async function () {  
  this.timeout(5000); // Mocha  
});  
  
jest.setTimeout(5000); // Jest
```

node:test

```
test("lent", { timeout: 5000 },  
  async () => {  
    await opLente();  
  }  
);
```

Règle : un timeout court force à mocker les I/O réelles.

Hooks setup / teardown

Préparer et nettoyer l'état autour des tests pour garantir l'isolation.

```
import { before, beforeEach, afterEach, after } from "node:test";

before(async () => { await db.connect(); });           // une fois, avant tout
beforeEach(async () => { await db.seed(); });           // avant chaque test
afterEach(async () => { await db.truncate(); });         // après chaque test
after(async () => { await db.close(); });               // une fois, à la fin
```

`beforeEach` / `afterEach` sont la clé de l'isolation : remise à zéro systématique.

2. Mocks et stubs

Doublures de test : le vocabulaire

Concept clé : remplacer une dépendance réelle par un objet contrôlé.

- **spy** : observe les appels sans changer le comportement
- **stub** : remplace une fonction par une réponse prédéfinie
- **mock** : stub + attentes vérifiées (appelé N fois, avec X)
- **fake** : implémentation simplifiée mais réelle

But : tester l'unité, pas ses dépendances (réseau, BDD, horloge).

sinon : spy et stub

```
import sinon from "sinon";

const spy = sinon.spy(logger, "info");
service.run();
assert.ok(spy.calledOnce);
assert.ok(spy.calledWith("démarré"));

const stub = sinon.stub(repo, "findById").resolves({ id: 1 });
const u = await service.get(1);
assert.equal(u.id, 1);

sinon.restore(); // dans afterEach : restaure les originaux
```

sinon : mock avec attentes

Le mock définit les attentes *avant*, puis on les vérifie avec `verify()`.

```
const mock = sinon.mock(mailer);
mock.expects("send")
  .once()
  .withArgs("user@test.io")
  .resolves(true);

await service.notify("user@test.io");

mock.verify(); // échoue si send n'a pas été appelé comme prévu
mock.restore();
```

Injection de dépendances

Concept clé : pour stubber facilement, ne pas instancier les dépendances en dur.

Difficile à tester

```
class Service {  
  constructor() {  
    this.repo = new Repo();  
  }  
}
```

Injectable

```
class Service {  
  constructor(repo, api) {  
    this.repo = repo;  
    this.api = api;  
  }  
}
```

On passe alors des fakes/stubs au constructeur dans les tests.

supertest : tester les routes Express

Lance l'app sans ouvrir de port réel, envoie de vraies requêtes HTTP simulées.

```
import request from "supertest";
import { app } from "../src/app.js";

test("GET /health renvoie 200", async () => {
  const res = await request(app).get("/health");
  assert.equal(res.status, 200);
  assert.deepEqual(res.body, { status: "ok" });
});

test("POST /users valide le corps", async () => {
  const res = await request(app).post("/users").send({});
  assert.equal(res.status, 400);
});
```

Démo

Voir `demos/demo-03.md`

Tester une route Express asynchrone avec supertest, isoler une dépendance externe avec un stub sinon, puis exécuter un test fonctionnel headless rapide.

3. Tests fonctionnels et frameworks

Tests fonctionnels : headless browsers

Concept clé : valider un parcours utilisateur réel dans un navigateur sans interface.

Playwright

- Multi-navigateurs (Chromium, Firefox, WebKit)
- Auto-wait intégré
- Runner et assertions inclus

Puppeteer

- Chromium/Chrome ciblé
- API bas niveau du protocole DevTools
- À combiner avec un runner externe

Plus lents que les tests unitaires : à réserver aux parcours critiques.

Exemple Playwright

```
import { test, expect } from "@playwright/test";

test("connexion réussie", async ({ page }) => {
  await page.goto("http://localhost:3000/login");
  await page.fill("#email", "user@test.io");
  await page.fill("#password", "secret");
  await page.click("button[type=submit]");
  await expect(page).toHaveURL(/dashboard/);
});
```

`headless` est le défaut : aucune fenêtre, idéal pour la CI.

Mocha vs Jest vs node:test

Critère	Mocha	Jest	node:test
Runner	oui (assert externe)	tout-en-un	natif (Node 18+)
Mocking	sinon	intégré	mock natif
Couverture	nyc/c8	intégrée	--experimental-test-coverage
Config	flexible	conventionnelle	zéro dépendance
Watch / parallèle	via options	intégré	intégré

Choisir son framework

Privilégier

- **Jest** : projet front/full-stack, mocking riche clé en main
- **Mocha** : besoin de flexibilité, stack à la carte
- **node:test** : zéro dépendance, projets légers et modernes

Constantes

- supertest pour les routes HTTP
- sinon (ou mock natif) pour isoler
- Playwright pour le fonctionnel

Travaux pratiques

Fichier : `tp/tp-03.md`

Couvrir une API Express avec tests unitaires, tests asynchrones, mocks/stubs et un test fonctionnel headless. Au choix Mocha ou Jest, avec les deux configurations fournies.

Durée : environ 1h30.

Récapitulatif

- Isolation = déterminisme : hooks `beforeEach` / `afterEach` et timeouts courts
- `async/await` pour l'asynchrone, `done` réservé aux callbacks
- sinon (spy/stub/mock) + injection de dépendances pour découpler
- supertest pour les routes, Playwright/Puppeteer pour le fonctionnel
- Mocha, Jest ou `node:test` selon le contexte projet

Cheatsheet : `cheatsheet/module-03.md`

Questions ?

Module suivant : Gestion de la performance
